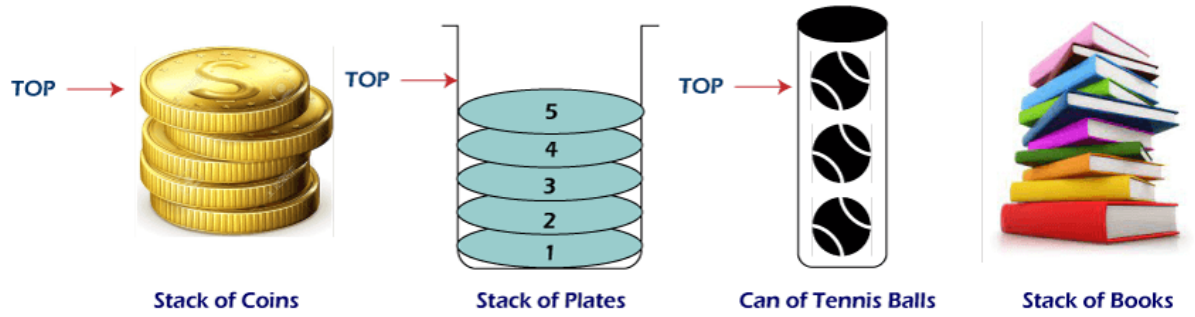
Module 7: Abstract Data Types (Week 4)

7.1 Stack

A Stack is an Abstract Data Type (ADT), commonly used in most programming languages. It is named stack as it behaves like a real-world stack such as a deck of cards or a pile of plates, etc. Other names for stacks are Piles and Push-down lists

A Stack is an ordered group of homogeneous items of elements. Elements are added to and removed from the top of the stack (the most recently added items are at the top of the stack). The last element to be added is the first to be removed. The Stack works on the principles of Last In, First Out (LIFO).

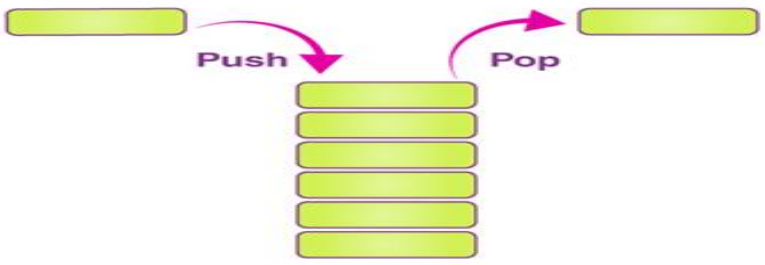


A Stack is a list of elements in which an element may be inserted or deleted only at one end, called TOP of the stack. The elements are removed in reverse order of that in which they were inserted into the stack. Stack uses pointers that always point to the topmost element within the stack, hence called as the top pointer.

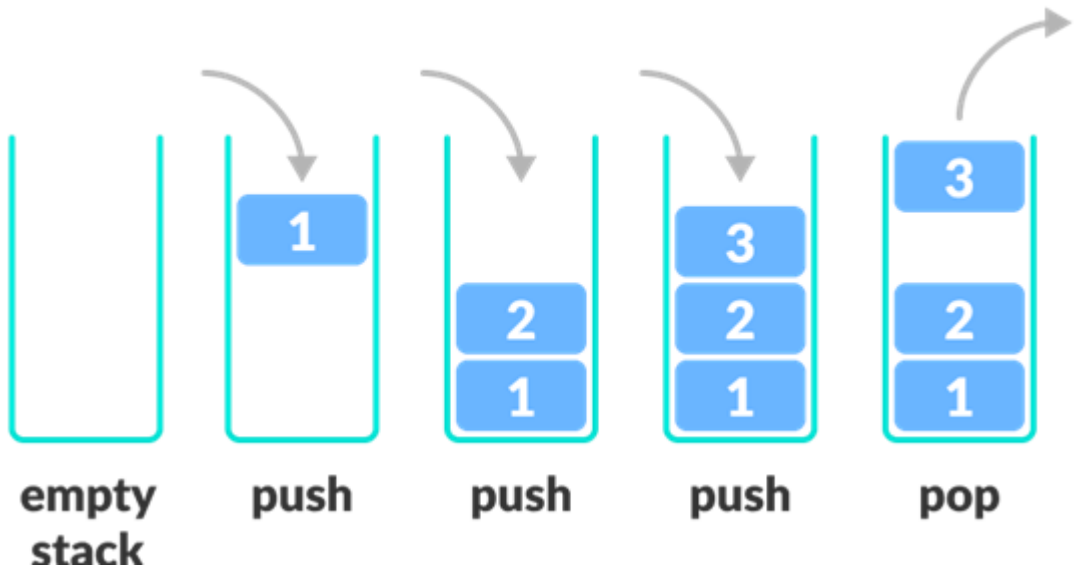
7.2 Stack Operations and Methods

Stack operations usually are performed for initialization, usage and de-initialization of the stack ADT. The two basic operations in the stack ADT include:

- push() : the term used to insert/add an element into a stack
- pop(): the term used to delete/remove an element from a stack.



In the diagram below, although item 3 was kept last, it was removed first. This is exactly how the LIFO (Last In First Out) Principle works.



Other Operations are:

- initialize(): creates/initializes the stack (i.e. create an empty Stack)
- isEmpty(): It determines whether the stack is empty or not.
- isFull(): It determines whether the stack is full or not.'
- peek(): It returns the element at the given position.
- count(): It returns the total number of elements available in a stack.
- change(): It changes the element at the given position.
- display(): It prints all the elements available in the stack.
- Size(): it determine the number of elements that are present inside the stack.
- Destroy(): deletes the contents of the stack (may be implemented by re-initializing the stack)

A Bounded stack is a stack has a fixed maximum capacity. If the stack is full and does not contain enough space to accept an entity to be pushed, the stack is then considered to be in an overflow state.

An Unbounded stack is a stack with no fixed capacity. It is better to implement an unbounded stack using a linked list. When we push and pop, we only need to add new node to the head or remove the head node, which cost O(1) for both.

7.3 Stack Overflow and Underflow

Stack is said to be in **Overflow** state when it is completely full and does not contain enough space to accept an entity to be pushed. A stack is said to be

in **Underflow** state if it is completely empty and does not contain any elements to be deleted.

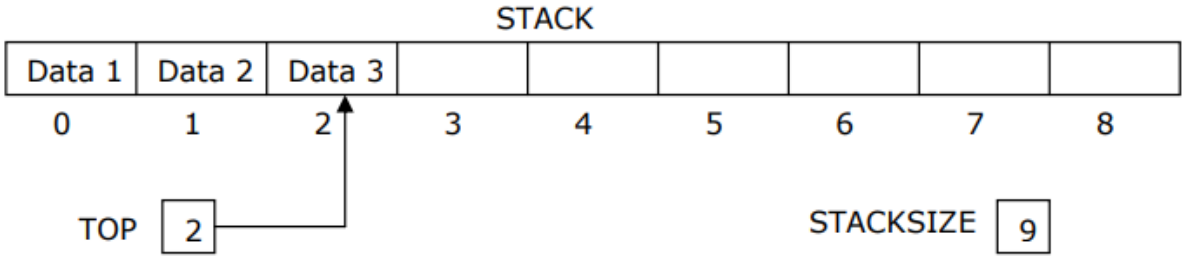
7.4 Stack Implementations in Data Structures

A Stack can be implemented by means of Array, Structure, Pointer, and Linked list. Stack can either be a fixed size one or it may have a sense of dynamic resizing.

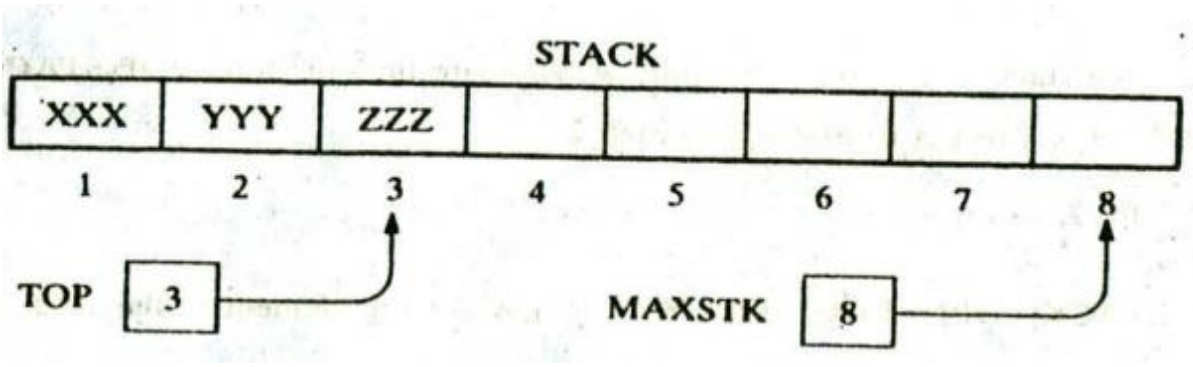
7.4.1 Array Implementation of Stack

Here, we are going to implement stack using arrays, which makes it a fixed size stack implementation.

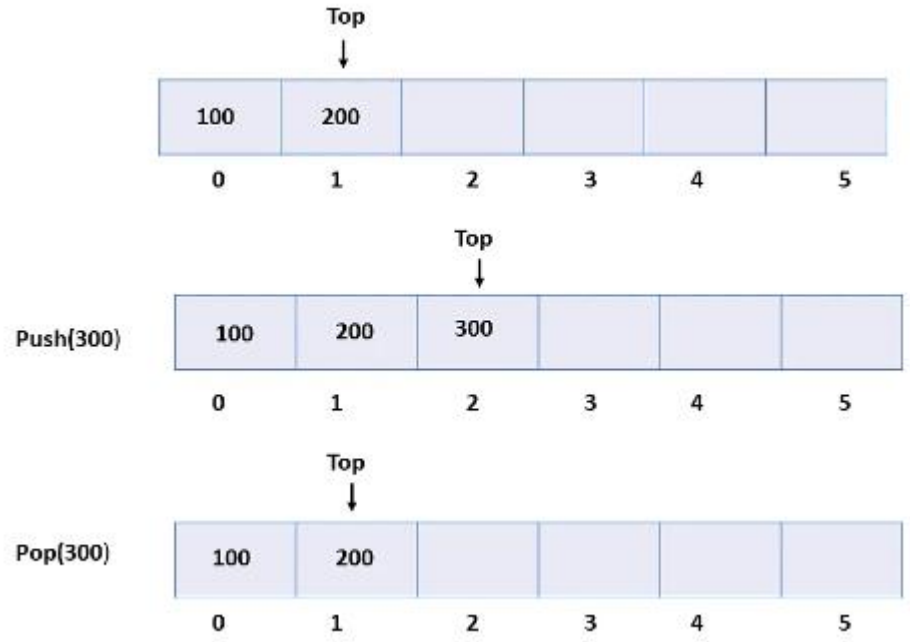
Let consider a linear array *STACK*, a variable *TOP* which contain the location of the top element of the stack; and a variable *STACKSIZE* which gives the maximum number of elements that can be hold by the stack. The condition TOP = 0 or TOP = Null will indicate that the stack is empty.



The Figure below represent an array representation of stack with three elements. Since TOP = 3, the stack has three demerits, XXX, YYY and ZZZ; and since MAXSTK = 8, there is room for 5 more items in the stack.

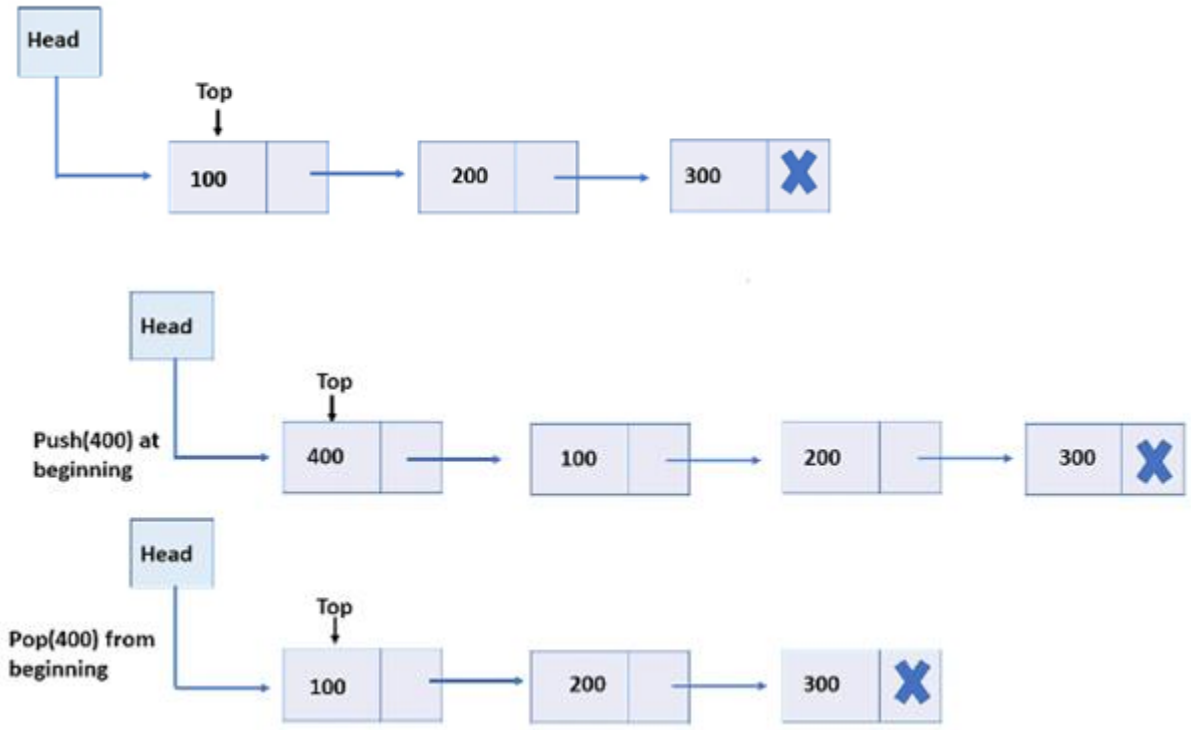


The way array is used to implemented stack operations is shown below:



7.4.2 Linked List Implementation of Stack

Every new element is inserted as a top element in the linked list implementation of stacks in data structures. That means every newly inserted element is pointed to the top. Whenever you want to remove an element from the stack, remove the node indicated by the top, by moving the top to its previous node in the list.



7.5 Basic Stack Operations Algorithm

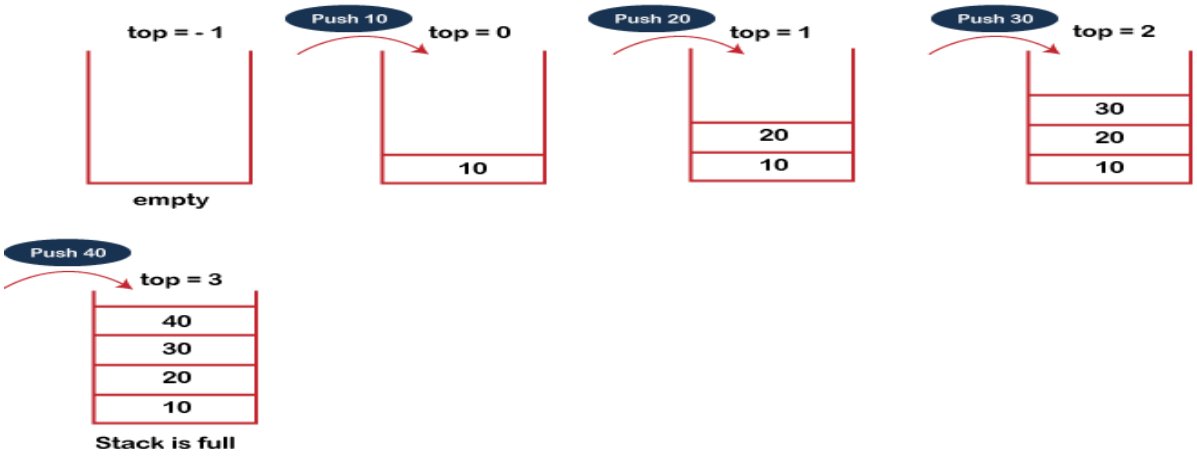
The operation of adding (pushing) an item onto a stack and the operation. of removing (popping) an item from a stack may be implemented, respectively, by the following procedures, called PUSH and POP.

7.5.1 Push Operation

push() is an operation that inserts elements into the stack. If the stack is full then the overflow condition occurs.

The steps involved in the PUSH operation is as follows:

Step 1: Check if the stack is full or not.
Step 2: If the stack is full, then print error of overflow and exit the program.
Step 3: If the stack is not full, then increment the top by 1 and add the element. **top=top+1,** and the element will be placed at the new position of the **top**. The elements will be inserted until we reach the ***max*** size of the stack.
The diagram below shows how the Push operation works:



The **algorithm** for the Push Operation is given below:

Algorithm: PUSH (STACK, TOP, STACKSIZE, ITEM)
[Algorithm to add an element to a Stack]

1. [STACK already filled?]
   If TOP=STACKSIZE-1, then: Print: OVERFLOW/Stack Full, and Return.
2. Set TOP: =TOP+1. [Increase TOP by 1.]
3. Set STACK[TOP]=ITEM. [Insert ITEM in new TOP position.]
4. RETURN.

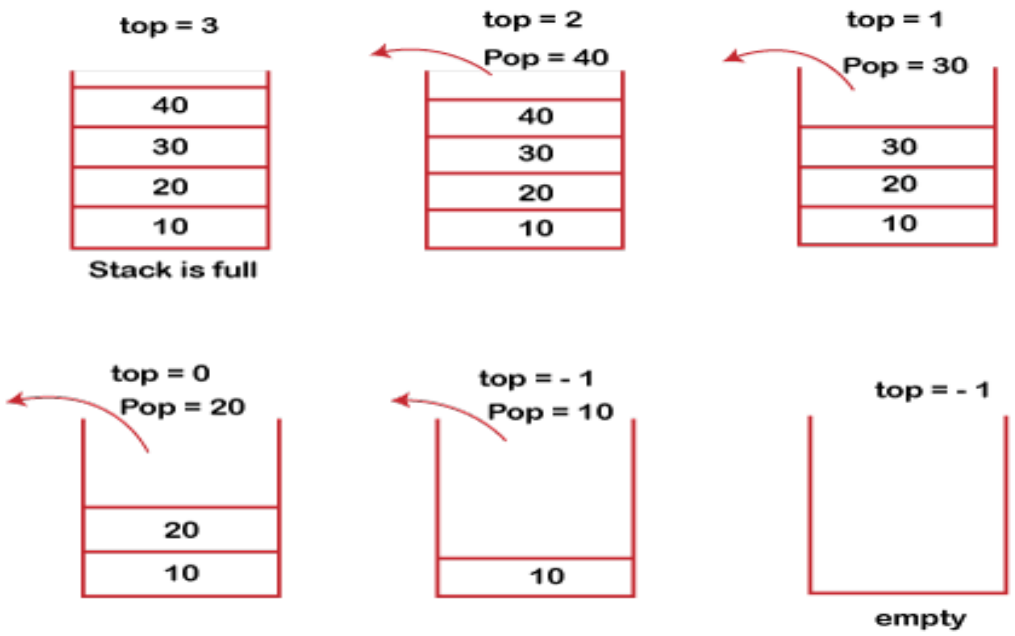
7.5.2 Pop Operations

pop() is a data manipulation operation which removes elements from the stack. If the stack is empty, it means that no element exists in the stack, this state is known as an underflow state.

The steps to pop an element from a Stack is as follows:

Step 1: Check if the stack is empty or not.
Step 2: If the stack is empty, then print error of underflow and exit the program.
Step 3: If the stack is not empty, then print the element at the top and decrement the top by 1. i.e., **top=top-1**.

The working of the pop operation of the stack is shown below:



7.6 Stack vs Basic Arrays

A Stack is a linear list data structure having a sequential collection of elements in which elements are added on the top of the stack like piles of books, etc. The new item can be added, or existing item can be removed only from the top of the stack. Stack is considered as a dynamic data structure because the size of the stack can be changed at the run time. The two operations that can be performed on the stack are push and pop. Here, push operation means that a new item is inserted into the stack, whereas the pop operation means that the existing item is removed from the stack. It strictly follows the LIFO (Last In, First Out) principle in which the recently added item

will be removed first, and the item which is added first will be removed last.

An Array is a form of linear data structure that is always defined as a collection of items that have the same data type. The value of the array is always stored at a place that has been predetermined and is referred to as the array's index. Arrays are not dynamic objects like stacks; rather, their sizes remain constant throughout their use. This means that once space is allocated for an array, its dimensions cannot be changed. Arrays are one of the efficient techniques to carry out computations of the same kind on a number of components that all belong to the same data type. Arrays can store one or more values that are of the same data type and allow access to those values using the indices that correspond to those values. Arrays are a type of data structure that provide random access, which means that the items are stored in a linear fashion but can be accessed randomly.

The differences between a Stack and an Array are shown in the table below:

Basis of comparison	**Stack**	**Array**
Definition	A stack is a type of linear data structure that is represented by a collection of pieces that are arranged in a predetermined sequence.	An array is a collection of data values that are associated to one another and termed elements. Each element is recognized by an indexed array.
Methods	Push, pop, and peek are the operations that one can perform on a stack.	It has a wide range of techniques or operations that can be applied to it, such as sorting, traversing, going backward, push, pop, etc.
Storage	The size of a stack is dynamic.	The size of an array is fixed.
Principle	The LIFO principle, which states that the element added last is the first to be removed from the list, is the foundation of stacks.	If you wish to get into the fourth element of the array, for example, you will need to specify the variable name together with its index or location within the square brackets, such as "arr[4]". This is because the items in the array are assigned to indices.
Data Access	Stacks do not permit arbitrary access to	In arrays, arbitrary access to elements is permitted.

	elements.	
Operations	When working with stacks, insertion and deletion can only take place from a single end of the list known as the top.	Any position in the array's index can serve as the starting point for an insertion or deletion operation.
Pointers	It has one pointer, and it points to the top. This pointer specifies the address of the element that is now at the top of the stack, which is also the element that was most recently added.	Memory can be allocated in arrays during build time, which is a process that is also referred to as static memory allocation.
Implementation	One can create a stack using an array.	Stacks cannot be used to implement an array.
Data Types	Elements of several data kinds may be found in a stack.	The items of an array are all of the same data type.

7.7 Arithmetic Expression: Polish Notations

Stacks are used by compilers to help in the process of converting infix to postfix arithmetic expressions and also evaluating arithmetic expressions. Arithmetic expressions consist of variables, constants, arithmetic operators and parentheses. The way to write arithmetic expression is known as a ***Notation***. An Arithmetic expression can be written in three different but equivalent notations, i.e., without changing the essence or output of an expression. These notations are:

- Infix Notation
- Prefix (Polish) Notation
- Postfix (Reverse-Polish) Notation

These notations are named the way the operator is used in an expression. To evaluate a complex infix expression, a compiler would first convert the expression to postfix notation, and then evaluate the postfix version of the expression.

When an operand is in between two different operators, which operator will take the operand first, is decided by the precedence of an operator over others. For example:

$$a + b * c \longrightarrow a + (b * c)$$

As multiplication operation has precedence over addition, b * c will be evaluated first

Associativity describes the rule where operators with the same precedence appear in an expression. For example, in expression $a + b - c$, both + and – have the same precedence, then which part of the expression will be evaluated first, is determined by associativity of those operators. Here, both + and − are left associative, so the expression will be evaluated as $(a + b) - c$. Precedence and associativity determines the order of evaluation of an expression.

We use the following three levels of precedence and associativity for the five binary operations.

S/NO	Operator	Precedence	Associativity
1	Exponentiation ^ or ↑	Highest	Right Associative
2	Multiplication (*) & Division (/)	Second Highest	Left Associative
3	Addition (+) & Subtraction (−)	Lowest	Left Associative

7.7.1 Infix Notation

When an operator appears between the two operands, it is called Infix notation. For example,

$$a - b + c$$

where operators are used in-between operands. With this notation, we must distinguish between *(A+B) *C* and *A+(B*C)* by using either parentheses or some Operator-precedence convention such as the usual precedence levels discussed above. Accordingly, the order of the operators and operands in an arithmetic expression does not uniquely determine the order in which the operations are to be performed. It is easy for us humans to read, write, and speak in infix notation but the same does not go well with computing devices. An algorithm to process infix notation could be difficult and costly in terms of time and space consumption.

Suppose we want to evaluate the following parenthesis-free arithmetic expression:

$$2\uparrow 3 + 5 * 2\uparrow 2 - 12/6$$

First, we evaluate the exponentiation to obtain

8 + 5 * 4 - 12 / 6

Then, we evaluate the multiplication and division to obtain 8 + 20— 2. Lastly, we evaluate the addition and subtraction to obtain the final result, 26. Observe that the expression is traversed three times, each time corresponding to a level of precedence of the operations.

7.7.2 Prefix (Polish) Notation

Polish notation, named after the Polish mathematician Jan Lukasiewicz, refers to the notation in which the operator is prefixed to operands, i.e. operator is written ahead of operands. For example, *+ab*. This is equivalent to its infix notation *a + b.* Polish notation is also known as Prefix notation. For example,

+AB, -CD, *EF and /GH

The following Infix Notations are translated into Polish notations using brackets [] to indicate a partial translation:

(A+B) * C=[+AB] * C= *+ABC
A +(B*C) = A + [BC]= +A*BC
(A + B)/(C - D) = [+AB)/[-CD] = /+AB-CD

The fundamental property of Polish notation is that the order in which the operations are to be performed is completely determined by the positions of the operators and operands in the expression. Accordingly, one never needs parentheses when writing expressions in Polish notation.

7.7.3 Postfix (Reverse-Polish) Notation

Reverse Polish/Postfix notation refers to the notation operator is postfixed to the operands i.e.; the operator is written after the two operands. For example,

AB+, CD-, EF*, GH/

Again, one never and parentheses to determine the order of the operations in any arithmetic expression written in reverse Polish notation. This notation is frequently called Postfix (or Suffix) notation.

The table below shows the difference in all three notations.

S/No.	Infix Notation	Prefix Notation	Postfix Notation
1	a + b	+ab	ab+
2	(a + b) ∗ c	∗+abc	ab+c∗
3	a ∗ (b + c)	∗a+bc	abc+∗
4	a/b + c/d	+/ab/cd	ab/cd/+
5	(a + b) ∗ (c + d)	∗+ab+cd	ab+cd+∗
6	((a + b) ∗ c) - d	-∗+abcd	ab+c∗d-
7	a + b * c + d	+ + a * b c d	abc*+ d+
8	a + b + c + d	+ + + a b c d	a b + c + d +

The computer usually evaluates an arithmetic expression written in infix notation in two Steps. First, it converts the expression to postfix notation, and then it evaluates the postfix expression. In each step, the stack is the main tool that is used to accomplish the given task. These applications of stacks will be implemented in reverse order. That is, first we show how stacks are used to evaluate posfix expression, and then we show how stacks are used to transform infix expressions into postfix expressions.

7.7.4 Evaluation of Postfix Expression

If P is an arithmetic expression written in postfix notation. This algorithm uses STACK to hold operands, and evaluate P.

Algorithm: This algorithm finds the VALUE of P written in postfix notation.

1. Add a Dollar Sign ”$” at the end of P. [This acts as sentinel.]
2. Scan P from left to right and repeat Steps 3 and 4 for each element of P until the sentinel “$” is encountered.
3. If an operand is encountered, put it on STACK.
4. If an operator © is encountered, then: [© represent an operator]
   (a) Remove the two top elements of STACK, where A is the top element and B is the next-to—top-element.
   (b) Evaluate B © A.
   (c) Place the result of (b) back on STACK.
   [End of If structure.]
   [End of Step 2 loop.]
5. Set VALUE equal to the top element on STACK.
6. Exit.

Example 1: Consider the following aarithmetic expression P written in postfix notation:

P: 5. 6. 2, +, *,12, 4, /, -

(Commas are used to separate the elements of P so that 5, 6, 2 is not interpreted as the number 562.)

Solution

Now add "$" at the end of expression as a sentinel.

*P: 5. 6. 2, +, *,12, 4, /, - $*

Symbol Scanned	Stack	Action to be taken
(1) 5	5	Push to Stack
(2) 6	5, 6	Push to Stack
(3) 2	5, 6,2	Push to Stack
(4) +	5,8	Remove the top elements, calculate 6 + 2 and push the result on stack
(5) *	40	Remove the top elements, calculate 5 + 8 and push the result on stack
(6) 12	40,12	Push to Stack
(7) 4	40,12, 4	Push to Stack
(8) /	40,3	Remove the top elements, calculate 12 + 4 and push the result on stack
(9) -	37	Remove the top elements, calculate 40 - 3 and push the result on stack
(10) $		Sentinel $ encountered, Result is on top of the STACK

The equivalent infix expression Q is as follows:

Q: 5 * (6 + 2) – 12/4

Note that parentheses are necessary for the infix expression Q but not for the postfix expression P.

We evaluate P by simulating Algorithm above. First, we add a sentinel right parenthesis at the end of P to obtain:

P:	5,	6,	2,	+,	*,	12,	4,	/,	-	$
	(1)	(2)	(3)	(4)	(5)	(6)	(7) (	8)	(9)	(10)

The elements of P have been labelled from left to right for easy reference. The table above shows the contents of the Stack as each element of P is scanned.

The final number in STACK. 37, which is assigned to VALUE when the sentinel "$" is scanned, is the value of P

7.7.5 Transforming Infix Expressions into Postfix Expressions

Let Q be an arithmetic expression written in infix notation. Besides operands and operators, Q may also contain left and right parentheses. We assume that the operators in Q consist only of exponentiation (^ or ↑), multiplications (*), divisions (/), additions (+) and subtractions (-), and that they have the usual three levels of precedence as given above. We also assume that Operators on the same level, including exponentiation are performed from left to right unless otherwise indicated by parentheses. (This is not a standard, since expressions may contain unary operators and some languages perform the exponentiations from right to left. However, these assumptions simplify our algorithm.)

The following algorithm transform the infix expression Q into its equivalent postfix expression P. It uses a stack to temporary hold the operators and left parenthesis. The postfix expression will be constructed from left to right using operands from Q and operators popped from STACK.

Algorithm: Infix_to_PostFix (Q, P)

[Suppose Q is an arithmetic expression written in infix notation. This algorithm finds the equivalent postfix expression P.]

1. Push "(" onto STACK, and add ")" to the end of Q.
2. Scan Q from left to right and repeat Steps 3 to 6 for each element of Q until the STACK is empty:
3. If an operand is encountered, add it to P.
4. If a left parenthesis is encountered, push it onto STACK.
5. If an operator © is encountered, then:
    (a) Repeatedly pop from STACK and add to P each operator (on top of STACK) which has the same or higher precedence/priority than ©
    (b) Add © to STACK.

    [End of If structure.]
6. If a right parenthesis is encountered, then:
    (a) Repeatedly pop from STACK and add to P each operator (on top of STACK) until a left parenthesis is encountered.
    (b) Remove the left parenthesis. [Do not add the left parenthesis to P.]

    [End of If structure.]

    [End of Step 2 loop.]
7. Exit.

Example 2: Convert P: A+ (B * C – (D / E ^ F) * G) * H into postfix form showing stack status .

Solution

Now add ")" at the end of expression A + (B * C – (D / E ^ F) * G) * H) and also Push a **"(“** on Stack.

Symbol Scanned	Stack	Expression P
A	(	A
+	(+	A
(	(+ (	A
B	(+ (	AB
*	(+ (*	AB
C	(+ (*	ABC
-	(+ (-	ABC*
(	(+ (- (	ABC*
D	(+ (- (	ABC*D
/	(+ (- (/	ABC*D
E	(+ (- (/	ABC*DE
^	(+ (- (/ ^	ABC*DE
F	(+ (- (/ ^	ABC*DEF
)	(+ (-	ABC*DEF^/
*	(+ (- *	ABC*DEF^/
G	(+ (- *	ABC*DEF^/G
)	(+	ABC*DEF^/G*-
*	(+ *	ABC*DEF^/G*-
H	(+ *	ABC*DEF^/G*-H
)	Empty	**ABC*DEF^/G*-H*+**

7.8 Complexity of Stack Operations

The time and space complexity of various stack operations are shown in the subsections.

7.8.1 Time Complexity

The time complexities for various operations that can be performed on the Stack data structure using the Array and Linked list implementation is shown in the table:

Stack Operations	Array Implementation	Linked List Implementation	Description
Push	O(1)	O(1)	The time complexities for push() function is O(1) because data has to be inserted from the **top** of the stack, which is a one step process.
Pop	O(1)	O(n)	For Array, the time complexities for pop() function is O(1) because data has to deleted from the **top** of the stack, which is a one step process.
Traversal	O(n)	O(n)	Displaying all the nodes of a stack needs traversing all the nodes of the linked list organized in the form of stack
Search	O(n)	O(n)	This involves the traversing all the lists
Peek ()	O(1)	O(1)	Only a memory address is accessed. This is a constant time operation.
isEmpty ()	O(1)	O(1)	It only performs an arithmetic operation to check if the stack is empty or not.
Size ()	O(1)	O(1)	This operation just performs a basic arithmetic operation.

7.8.2 Space Complexity

The space complexities for various operations that can be performed on the Stack data structure using the Array and Linked list implementation is shown in the table:

Stack Operations	Array Implementation	Linked List Implementation	Description
Push	O(1)	O(1)	As no extra space is being used

Pop	O(1)	O(1)	No extra space is utilized for deleting an element from the stack.
Traversal	O(n)	O(1)	No extra space is utilized to access the value but element has to traversed when using an Array.
Search	O(n)	O(1)	It requires no extra space but element has to traversed when using an Array.
Peek ()	O(1)	O(1)	No extra space is utilized to access the element because only the value in the node at the top pointer is read.
isEmpty ()	O(1)	O(1)	It requires no extra space.
Size ()	O(1)	O(1)	No extra space is required to calculate the value of the top pointer.

7.9 Applications of Stack

Although stack is a simple data structure to implement, it is very powerful. The most common Applications of a Stack Data Structures are:

(a) Evaluation of Arithmetic Expressions: A stack is a very effective data structure for evaluating arithmetic expressions in programming languages. An arithmetic expression consists of operands and operators. In addition to operands and operators, the arithmetic expression may also include parenthesis like "left parenthesis" and "right parenthesis".

(b) Backtracking: Backtracking is a recursive algorithm mechanism that is used to solve optimization problems. To solve the optimization problem with backtracking, there are must be multiple solutions and it does not matter if it is correct. While finding all the possible solutions in backtracking, the previously calculated problems are stored in the stack and that solution is used to resolve the following issues. The N-queen problem is an example of backtracking, a recursive algorithm where the stack is used to solve this problem.

(c) Delimiter Checking: The common application of Stack is delimiter checking, i.e., parsing that involves analyzing a source program syntactically. It is also called parenthesis checking. When the compiler translates a source program written in some programming language

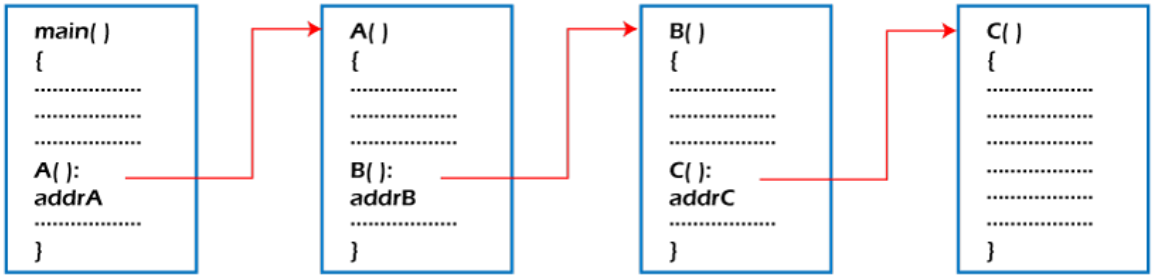
such as C, C++ to a machine language, it parses the program into multiple individual parts such as variable names, keywords, etc. By scanning from left to right. The main problem encountered while translating is the unmatched delimiters. We make use of different types of delimiters include the parenthesis checking (,), curly braces {,} and square brackets [,], and common delimiters /* and */. Every opening delimiter must match a closing delimiter, i.e., every opening parenthesis should be followed by a matching closing parenthesis. Also, the delimiter can be nested. The opening delimiter that occurs later in the source program should be closed before those occurring earlier. To perform a delimiter checking, the compiler makes use of a stack. When a compiler translates a source program, it reads the characters one at a time, and if it finds an opening delimiter it places it on a stack. When a closing delimiter is found, it pops up the opening delimiter from the top of the Stack and matches it with the closing delimiter.

(d) Reverse a Data: To reverse a given set of data, we need to reorder the data so that the first and last elements are exchanged, the second and second last element are exchanged, and so on for all other elements. There are different reversing applications are reversing a string and converting Decimal to Binary.

Stack can be used to reverse the characters of a string. This can be achieved by simply pushing one by one each character onto the Stack, which later can be popped from the Stack one by one. Because of the last in first out property of the Stack, the first character of the Stack is on the bottom of the Stack and the last character of the String is on the Top of the Stack and after performing the pop operation in the Stack, the Stack returns the String in Reverse order.

Although decimal numbers are used in most business applications, some scientific and technical applications require numbers in either binary, octal, or hexadecimal. A stack can be used to convert a number from decimal to binary/octal/hexadecimal form. For converting any decimal number to a binary number, we repeatedly divide the decimal number by two and push the remainder of each division onto the Stack until the number is reduced to 0. Then, we pop the whole Stack and the result obtained is the binary equivalent of the given decimal number.

(e) Processing Function Calls: Stack plays an important role in programs that call several functions in succession. Suppose we have a program containing three functions: A, B, and C. function A invokes function B, which invokes the function C.



Function call

When we invoke function A, which contains a call to function B, then its processing will not be completed until function B has completed its execution and returned. Similarly for function B and C. Therefore, we observe that function A will only be completed after function B is completed and function B will only be completed after function C is completed. Therefore, function A is first to be started and last to be completed. To conclude, the above function activity matches the last in first out behaviour and can easily be handled using Stack.

(f) Browsers: The back button in a browser saves all the URLs that have been visited previously in a stack. Each time a new page is visited, it is added on top of the stack. When the back button is pressed, the current URL is removed from the stack, and the previous URL is accessed.

(g) Syntax Parsing: Since many programming languages are context-free languages, the stack is used for syntax parsing by many compilers.

(h) Memory Management: Memory management is an essential feature of the operating system, so the stack is heavily used to manage memory.

7.10 Hands-on Submissions on Stack Methods

Implement all the stack operations in C++ or Java.